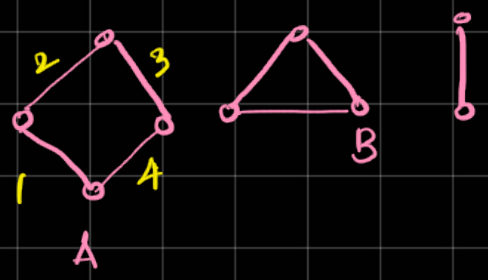
Disjoint Sets

→ collection of sets

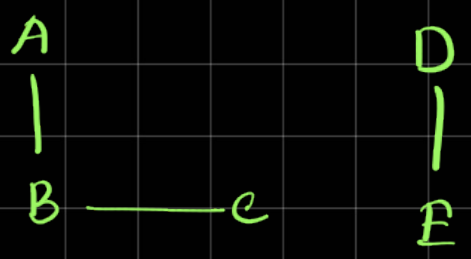
Kruskal's algo.



minimum spanning tree

find connected nodes.

A B C D E



- Find connected components in a graph/
- whether the graph contains any cycle or not.
- minimum spanning tree.

4 points to Note:

① Those who are connected/friend to each other will belong to the same set.

② # of Disjoint set represents → # of subgraphs.

③ Every set will have a "representative".

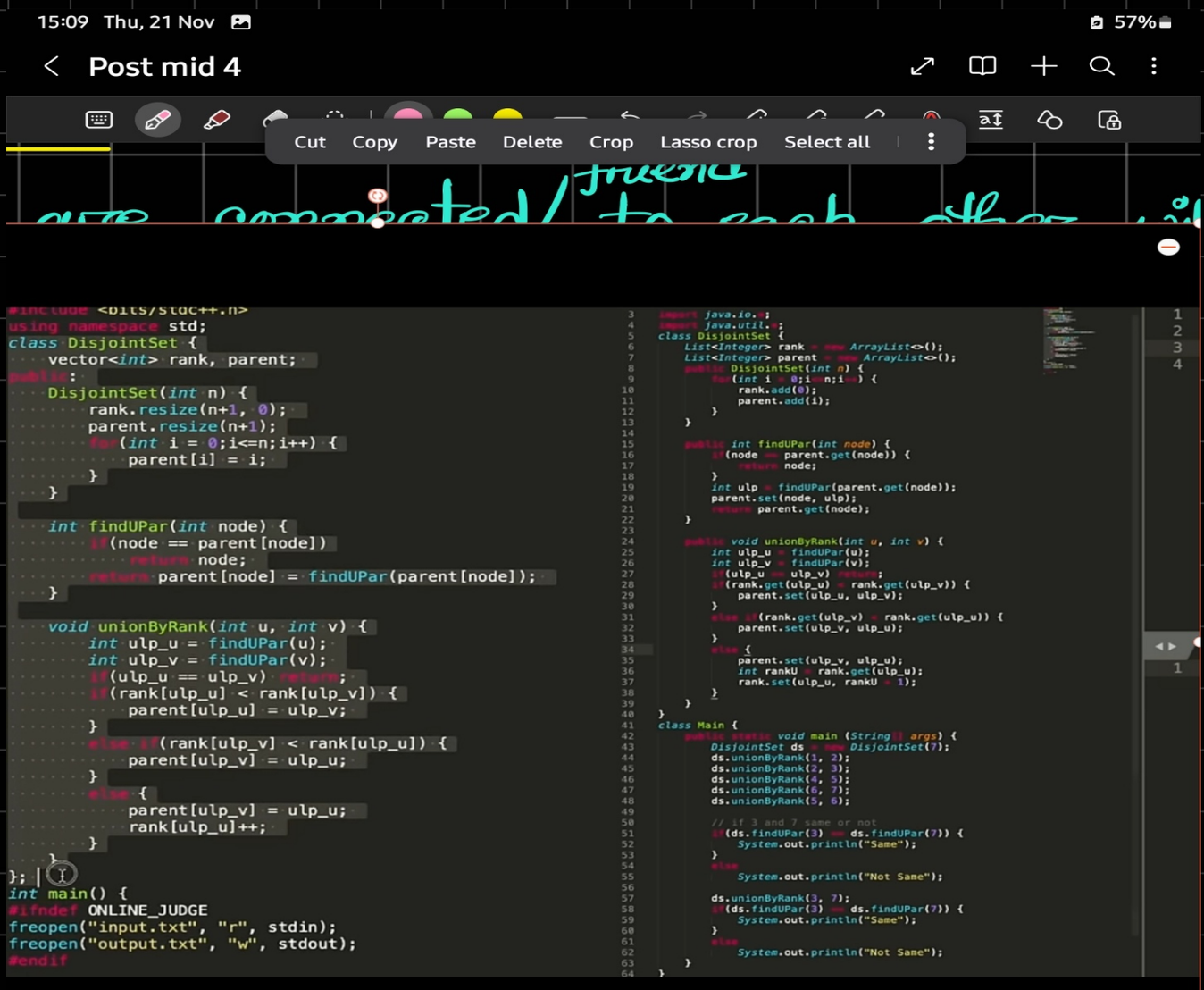
④ If two items have the same representative, They belong to the same set.

3 operations:

(i) make set → Create separate sets, with each item (initialization)

(ii) Find → return the representative

(iii) Union → Connect the representatives of two sets & form a new set.




```cpp
#include <bits/stdc++.h>
using namespace std;
class DisjointSet {
    vector<int> rank, parent;
public:
    DisjointSet(int n) {
        rank.resize(n+1, 0);
        parent.resize(n+1);
        for(int i = 0;i<=n;i++) {
            parent[i] = i;
        }
    }

    int findUPar(int node) {
        if(node == parent[node])
            return node;
        return parent[node] = findUPar(parent[node]);
    }

    void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if(ulp_u == ulp_v) return;
        if(rank[ulp_u] < rank[ulp_v]) {
            parent[ulp_u] = ulp_v;
        }
        else if(rank[ulp_v] < rank[ulp_u]) {
            parent[ulp_v] = ulp_u;
        }
        else {
            parent[ulp_v] = ulp_u;
            rank[ulp_u]++;
        }
    }
};
int main() {
#ifndef ONLINE_JUDGE
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
#endif
```

```java
import java.io.*;
import java.util.*;
class DisjointSet {
    List<Integer> rank = new ArrayList<>();
    List<Integer> parent = new ArrayList<>();
    public DisjointSet(int n) {
        for(int i = 0;i<=n;i++) {
            rank.add(0);
            parent.add(i);
        }
    }

    public int findUPar(int node) {
        if(node == parent.get(node)) {
            return node;
        }
        int ulp = findUPar(parent.get(node));
        parent.set(node, ulp);
        return parent.get(node);
    }

    public void unionByRank(int u, int v) {
        int ulp_u = findUPar(u);
        int ulp_v = findUPar(v);
        if(ulp_u == ulp_v) return;
        if(rank.get(ulp_u) < rank.get(ulp_v)) {
            parent.set(ulp_u, ulp_v);
        }
        else if(rank.get(ulp_v) < rank.get(ulp_u)) {
            parent.set(ulp_v, ulp_u);
        }
        else {
            parent.set(ulp_v, ulp_u);
            int rankU = rank.get(ulp_u);
            rank.set(ulp_u, rankU + 1);
        }
    }
}
class Main {
    public static void main (String[] args) {
        DisjointSet ds = new DisjointSet(7);
        ds.unionByRank(1, 2);
        ds.unionByRank(2, 3);
        ds.unionByRank(4, 5);
        ds.unionByRank(6, 7);
        ds.unionByRank(5, 6);

        // if 3 and 7 same or not
        if(ds.findUPar(3) == ds.findUPar(7)) {
            System.out.println("Same");
        }
        else
            System.out.println("Not Same");

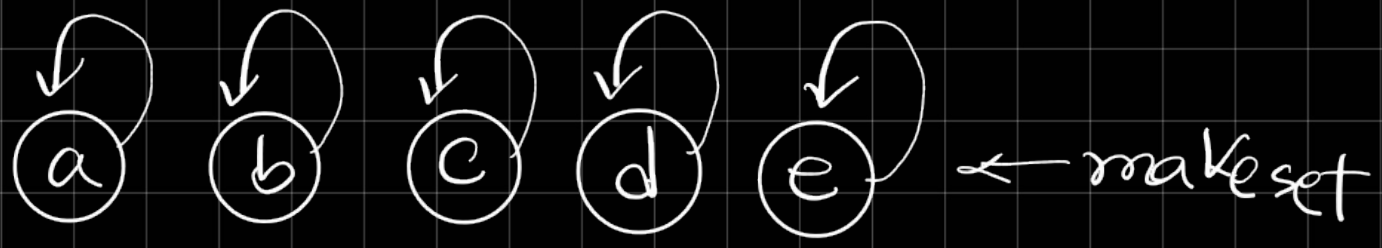
        ds.unionByRank(3, 7);
        if(ds.findUPar(3) == ds.findUPar(7)) {
            System.out.println("Same");
        }
        else
            System.out.println("Not Same");
    }
}
```

TUF

et

make set:

a b c d e

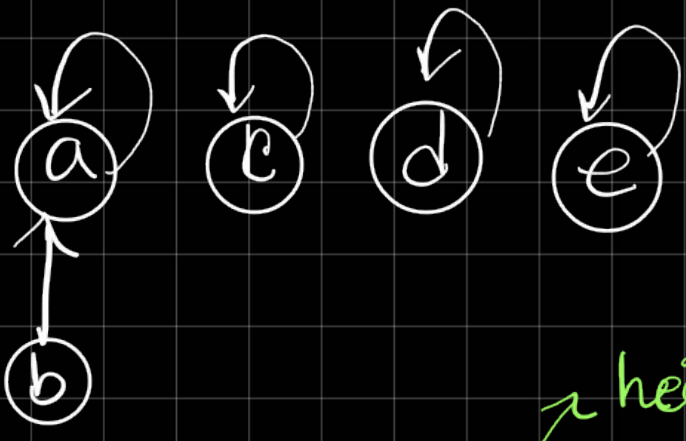


→ is a, b connected?

Find (a) = a
Find (b) = b
} a, b: disjoint.

→ Union (a, b)

Find (a) = a
Find (b) = b



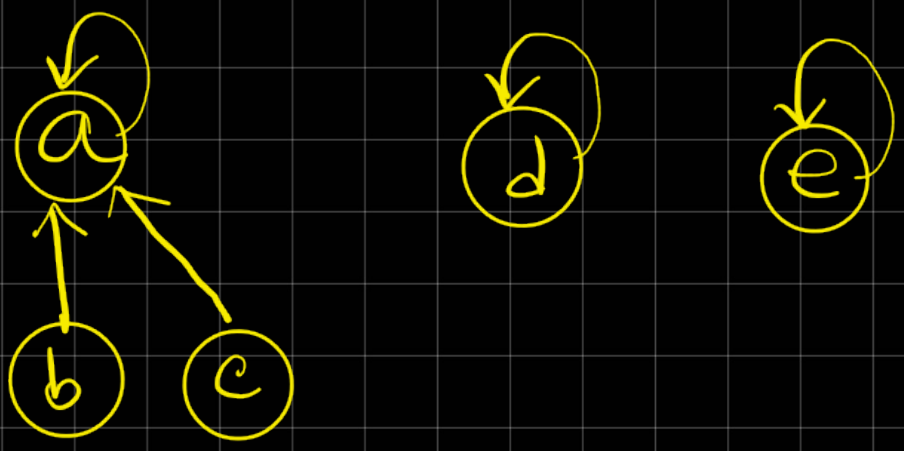
Union (c, a) — [Union by rank (↗ height.)
" " size

Find (c) = c

Find (a) = a

if a gets under c find operation

costly go up → up ....



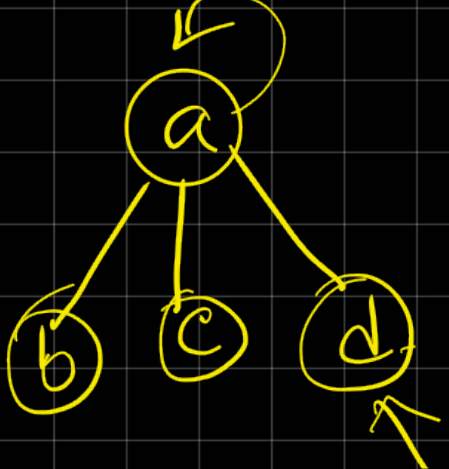
Union (d, e)



Union (c, e)

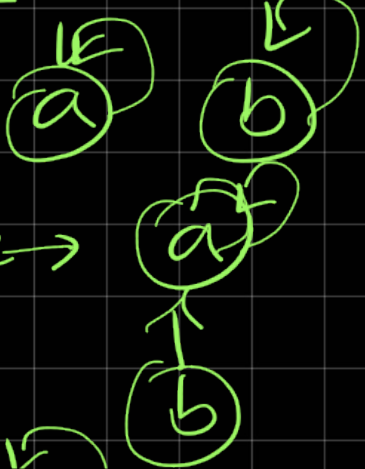
Find(c) = a

Find(e) = d



When does rank increase → when size is same

as



same →



not same size

So rank not increased.

redundant edge → cycle.

Yaay.

```
makeset(  )
    for (i=0 ——— N)
    parent[i] = i
```

```
int Find(u)
    if parent[u] == u
        return u
    else
        parent[u] = Find(parent[u])
        return parent[u]
```

path compression

assign & return

union (u, v)

f(e) ↖

f(d) ↗

r_1 = Find (u)

r_2 = Find (v)

f(a)

if $r_1 \neq r_2$

~~parent $[r_2] = r_1$~~

if (rank $[r_1]$ > rank $[r_2]$)

parent $[r_2] = r_1$

else if (rank $[r_2]$ > rank $[r_1]$)

parent $[r_1] = r_2$

else

parent $[r_2] = r_1$

rank $[r_1]$ ++

parent/reps rank++

cycle detection algo → redundant edge:

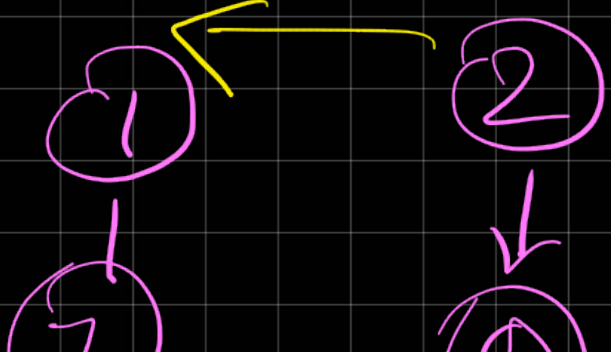
if we get a new set where 2 sets were already in a set.

---

u(i, j)
u(5, 7)

r_i = find(5) = 1
r_j = " (7) = 7

✗ Parent(j) = r_i → problem
parent(r_j) = r_i



④ ⑤

$\cup(7,0)$

$r_i = 1$

$r_j = 2$

$p[2] = 1$

Maze Generation